

NAME:Baseera bibi

REG NO:Sp24-bse-013

Lab1

TASK1:

Design a Java class named CourseResult containing studentName, courseName, and grade attributes. The class should provide a display() method to display these values. A runner class creates two CourseResult objects.

.CourseResult.java

```
public class CourseResult {  
  
    public String studentName;  
    public String courseName;  
    public String grade;  
  
    public void display() {  
        System.out.println("Student Name is: " + studentName);  
        System.out.println("Course Name is: " + courseName);  
        System.out.println("Grade is: " + grade);  
    }  
}
```

CourseResultRun.java.

```
public class CourseResultRun {  
  
    public static void main(String[] args) {  
  
        CourseResult c1 = new CourseResult();  
  
        c1.studentName = "Ali";  
        c1.courseName = "OOP";  
        c1.grade = "A";  
  
        c1.display();  
  
        CourseResult c2 = new CourseResult();  
  
        c2.studentName = "Saba";  
        c2.courseName = "ICP";  
        c2.grade = "A+";  
  
        c2.display();  
    }  
}
```

Execution

Compile: javac CourseResult.java CourseResultRun.java

Run: java CourseResultRun

SCREEN SHOT

```
Student Name is: Ali
Course Name is: OOP
Grade is: A
Student Name is: Saba
Course Name is: ICP
Grade is: A+
```

Task 2

Point and Line Classes Create a Point class with x and y coordinates, a parameterized constructor, setters, getters, and a display() method. Create a Line class containing startPoint and endPoint and calculate the line length using the Euclidean distance formula. Distance Formula

Point.java

```
public class Point {
    private double x;
    private double y;
    public Point(double x, double y) {
        this.x = x;
        this.y = y;
    }
    public void setX(double x)
    { this.x = x; }
    public void setY(double y)
    { this.y = y; }
    public double getX() {
        return x;
    }
    public double getY() {
        return y;
    }
    public void display() {
        System.out.println("Point: (" + x + ", " + y + ")");
    }
}
```

Line.java

```
public class Line {

    private Point startPoint;
    private Point endPoint;

    public Line(Point startPoint, Point endPoint) {
        this.startPoint = startPoint;
        this.endPoint = endPoint;
    }

    public double findLength() {
```

```

        double x1 = startPoint.getX();
        double y1 = startPoint.getY();

        double x2 = endPoint.getX();
        double y2 = endPoint.getY();

        return Math.sqrt(
            Math.pow(x2 - x1, 2) +
            Math.pow(y2 - y1, 2)
        );
    }

    public void displayLength() {
        System.out.println("Line Length = " + findLength());
    }
}

```

LineRunner.java

```

public class LineRunner {

    public static void main(String[] args) {

        Point p1 = new Point(0, 0);
        Point p2 = new Point(3, 4);

        Line line1 = new Line(p1, p2);

        line1.displayLength();

        Point p3 = new Point(1, 1);
        Point p4 = new Point(4, 5);

        Line line2 = new Line(p3, p4);

        line2.displayLength();
    }
}

```

Execution

Compile: `javac CourseResult.java CourseResultRun.java`

Run: `java CourseResultRun`

SCREEN SHOT

```

Line Length = 5.0
Line Length = 5.0

```

Test Cases

1. Non-Failure Test Cases

Test Case	Points	Expected Length	Actual Length	Status
TC-01	(0,0), (3,4)	5	5	PASS
TC-02	(1,1), (1,6)	5	5	PASS
TC-03	(2,3), (2,3)	0	0	PASS

2. Failure Test Cases

Test Case	Fault Introduced	Expected Result	Faulty Result	Status
TC-04	Use $x + x$ instead of $x - x$	$\sqrt{13} \approx 3.606$	5	FAIL
TC-05	Use $y + y$ instead of $y - y$	$\sqrt{18} \approx 4.243$	5	FAIL
TC-06	Remove square operations	$\sqrt{7} \approx 2.646$	$\sqrt{58} \approx 7.616$	FAIL

3. Testing Observation

The non-failure test cases verify that the correct implementation produces the expected line lengths for different input points, including normal, vertical, and zero-length lines.

The failure test cases demonstrate that when faults are intentionally introduced into the source code, the externally observed results change from the expected values. This confirms that the test cases are capable of detecting implementation errors.

Important: After each fault-based test, the source code should be restored to its correct version before running the next test case.

Test Summary

Category	Total Test Cases	PASS	FAIL
Non-Failure Tests	3	3	0
Failure Tests	3	0	3
Total	6	3	3

Conclusion: The testing demonstrates that the correct implementation passes all non-failure test cases, while the fault-based test cases successfully identify the introduced faults.

Task 3:

Defect Identification and Reporting in CUOnline The CUOnline system can be tested by examining major functional areas such as authentication, course registration, academic results, and student profile management. Any unexpected behavior should be documented in a structured defect report

Test Cases

1. Non-Failure Test Cases

Test Case	Points	Expected Length	Actual Length	Status
TC-01	(0,0), (3,4)	5	5	PASS
TC-02	(1,1), (1,6)	5	5	PASS
TC-03	(2,3), (2,3)	0	0	PASS

2. Failure Test Cases

Test Case	Fault Introduced	Expected Result	Faulty Result	Status
TC-04	Use $x + x$ instead of $x - x$	$\sqrt{13} \approx 3.606$	5	FAIL
TC-05	Use $y + y$ instead of $y - y$	$\sqrt{18} \approx 4.243$	5	FAIL
TC-06	Remove square operations	$\sqrt{7} \approx 2.646$	$\sqrt{58} \approx 7.616$	FAIL

3. Testing Observation

The non-failure test cases verify that the correct implementation produces the expected line lengths for different input points, including normal, vertical, and zero-length lines.

The failure test cases demonstrate that when faults are intentionally introduced into the source code, the externally observed results change from the expected values. This confirms that the test cases are capable of detecting implementation errors.

Important: After each fault-based test, the source code should be restored to its correct version before running the next test case.

4. Test Summary

Category	Total Test Cases	PASS	FAIL
Non-Failure Tests	3	3	0
Failure Tests	3	0	3
Total	6	3	3

Conclusion

The testing demonstrates that the correct implementation passes all non-failure test cases, while the fault-based test cases successfully identify the introduced faults. Therefore, the test cases are effective in verifying the correctness of the `Line` class implementation and detecting changes caused by intentionally introduced faults.